

Lecture 7: Convolutional Neural Networks

Yaxin Bi
School of Computing
Ulster University

1

Contents

- ▶ Convolutional Neural Networks
- ▶ Convolutional Layer
- ▶ Pooling Layer
- ▶ Full Connection Layer
- ▶ Other Convolutional Neural Networks

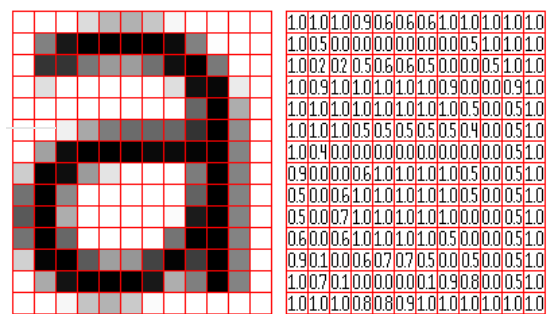
2

Convolutional Neural Networks

- ▶ A Convolutional Neural Network, also known as CNN or ConvNet, which specializes in processing data that has a grid-like topology, such as an image.
- ▶ A digital image is a visual representation of data or scene
 - ▶ a series of pixels arranged in a grid-like fashion
 - ▶ pixel values to denote how bright and what color
- ▶ Each neuron works in its own receptive field and is connected to other neurons in a way, they cover the entire visual field.
- ▶ A **receptive field** is the restricted region of the visual field, in which each neuron processes data only
- ▶ The layers are arranged in such a way so that they detect simpler patterns first (lines, curves, etc.) and more complex patterns

3

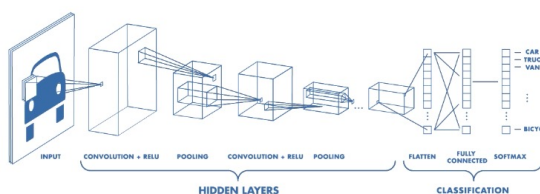
Representation of Image as a Grid of Pixels



4

Architecture of CNN

- ▶ A CNN typically has three layers: a convolutional layer; a pooling layer; and a fully connected layer.



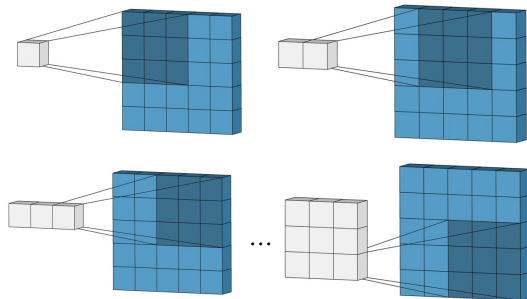
5

Convolutional Layer

- ▶ The convolution layer is the core building block of the CNN. It carries the main portion of the network's computational load.
- ▶ This layer performs a **dot product** between two matrices,
 - ▶ one matrix is the set of learnable parameters, known as a kernel
 - ▶ the other matrix is the restricted portion of the receptive field
- ▶ The kernel is spatially smaller than an image but is more in-depth.
 - ▶ if the image is composed of three (RGB) channels, the kernel height and width will be spatially small,
 - ▶ but the depth extends up to all three channels.

6

An Illustration of CNN Operation



7

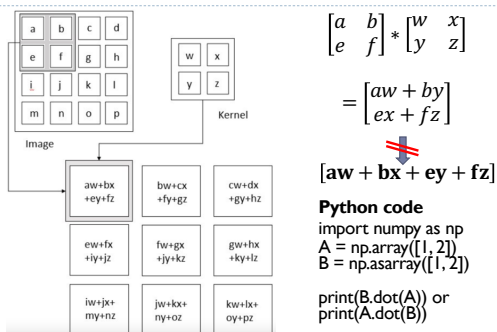
CNN Operation

- ▶ With the forward pass, the kernel slides across the height and width of the image producing the image representation of that receptive region.
- ▶ Resulting a two-dimensional representation of the image known as an activation map that gives the response of the kernel at each spatial position of the image.
 - ▶ The sliding size of the kernel is called a **stride**.
- ▶ If we have an input of size $V \times V \times D$ and D_{out} number of kernels with a spatial size of F with stride S and amount of padding P
- ▶ then the size of output volume can be determined by the following formula:

$$W_{out} = \frac{W - F + 2P}{S} + 1$$

8

Convolution Operation (cont'd)



9

Motivation behind Convolution

- ▶ Convolution leverages three important ideas:
 - ▶ sparse interaction
 - ▶ parameter sharing
 - ▶ equivariant representation
- ▶ CNN layers use matrix multiplication by a matrix of parameters describing the interaction between the input and output unit.
 - ▶ every output unit interacts with every input unit.
- ▶ For sparse interaction, it is achieved by making a kernel smaller than the input
 - ▶ For example an image can have millions or thousands of pixels, using kernel to process the image, detecting meaningful information of tens or hundreds of pixels.

10

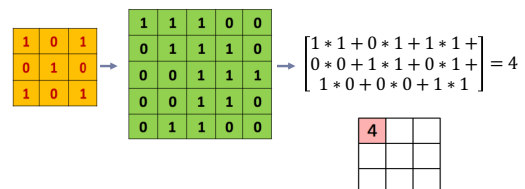
Motivation behind Convolution (cont'd)

- ▶ For a single two-dimensional slice i.e., for creating one activation map, neurons are constrained to use the same set of weights
- ▶ However in a traditional neural network,
 - ▶ each element of the weight matrix is used once and then never revisited,
 - ▶ while convolution network has shared parameters i.e., for getting output, weights applied to one input are the same as the weight applied elsewhere.
- ▶ Due to parameter sharing, the layers of CNN will have a property of equivariance to translation
 - ▶ if we changed the input in a way, the output will also get changed in the same way.

11

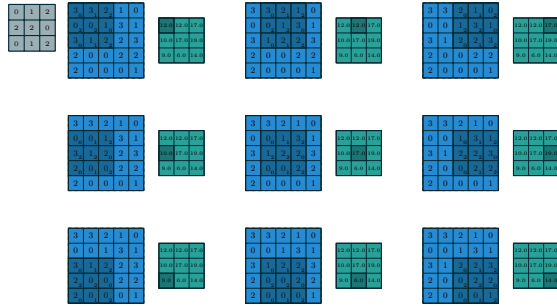
Computing Values of a Convolution

- ▶ The green grid is called the input feature map, but it is common to have multiple feature maps produced one onto another. The yellow grid is called a kernel

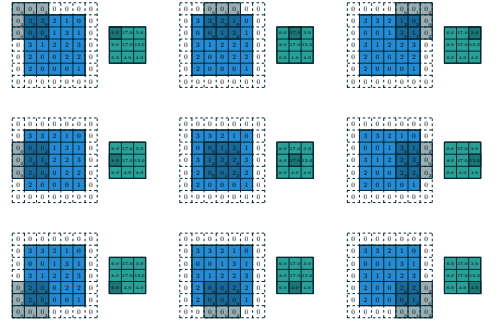


12

Computing Values of a Convolution (cont'd)



13

Computing Values of a Convolution (cont'd)
with padding=1, stride=2

14

Pooling Layer

- ▶ The pooling layer replaces the output of the network at certain locations by deriving a summary statistic of the nearby outputs.
 - ▶ reducing the spatial size of the representation
 - ▶ decreasing the required amount of computation and weights.
- ▶ The pooling operation is processed on every slice of the representation individually.
- ▶ There are several pooling functions
 - ▶ average of the rectangular neighbourhood
 - ▶ L2 norm of the rectangular neighborhood,
 - ▶ weighted average based on the distance from the central pixel.
 - ▶ the maximum output from the neighborhood.

15

Pooling Layer (cont'd)

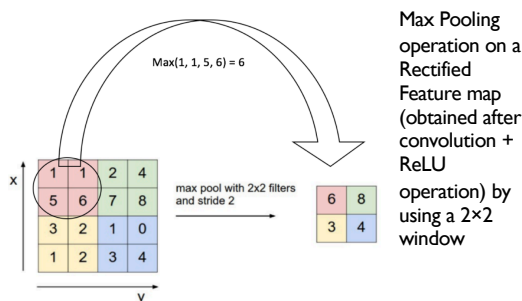
- ▶ If we have an activation map of size $W \times W \times D$, a pooling kernel of spatial size F , and stride S , then the size of output volume can be determined by the following formula:

$$W_{out} = \frac{W - F}{S} + 1$$

- ▶ This will yield an output volume of size $W_{out} \times W_{out} \times D$.
- ▶ Pooling provides some translation invariance
 - ▶ an object would be recognizable regardless of where it appears on the frame.

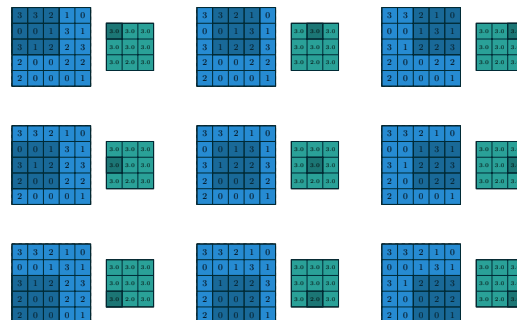
16

Max Pooling



17

Example: max pooling on a 5x5 input using 1x1 strides



18

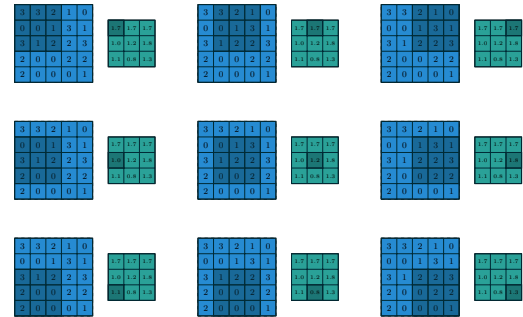
Pooling Layer (cont'd)

- Pooling operations make up another important building block in CNNs
 - reduce the size of feature maps by using some functions
- Pooling works by sliding a window across the input and feeding the content of the window to a pooling function.
- In some sense, pooling works very much like a convolution, but replaces the linear combination described by the kernel with some other function.
- The next slide provides an example for average pooling
 - A 3×3 average pooling operation on a 5×5 input using 1×1 strides.

19

19

Example: average pooling on a 5×5 input



20

20

Stride and Padding

- Stride is the number of pixels by which we slide a kernel matrix over the input matrix.
 - when stride = 1, move the kernel one pixel at a time;
 - when stride = 2, the kernel jump two pixels at a time.
 - a larger stride will produce smaller feature maps.
- Zero-padding is to pad the input matrix with zeros around the border, so that we can apply the kernel to bordering elements of an input image matrix.
 - zero padding allows to control the size of the feature maps.
 - adding zero-padding is also called *wide convolution*, so naturally not using zero-padding would be a *narrow convolution*.

21

21

Fully Connected Layer

- The Fully Connected (FC) layer is a traditional Multi-layer Perceptron that uses a **softmax** activation function in the output layer
- It implies that every neuron in the previous layer is connected to every neuron on the next layer.
- The output from the convolutional and pooling layers represent high-level features of the input image.
 - map the representation between the input and the output.
- The purpose of the FC layer is to use these features for classifying the input image into various classes based on the training dataset.

22

22

Fully Connected Layer (cont'd)

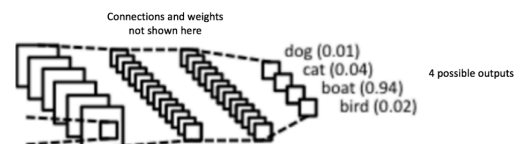
- Apart from classification, adding a fully-connected layer is also a (usually) cheap way of learning non-linear combinations of these features.
- Most of the features from convolutional and pooling layers may be good for the classification task, but combinations of those features might be even better
- The sum of output probabilities from the Fully Connected Layer is 1.
- This is ensured by using the Softmax as the activation function in the output layer of the FC Layer.

23

23

Fully Connected Layer (cont'd)

- The FC Layer makes each neuron to be connected to every other neuron in the adjacent layer
- For example, the image classification task is to classify images into four possible outputs as shown below (note that this does not show connections between the neurons in the fully connected layer)



24

24

Example: 1D Convolutional Layer

- Apply the single kernel to our input data, detect bump

$[0, 1, 0]$
 $[0, 0, 0, 1, 1, 0, 0, 0]$
 Compute dot product:
 $(0 \times 0) + (1 \times 0) + (0 \times 0) = 0$
 $[0, 1, 0] \cdot [0, 0, 0] = 0; [0, 1, 0] \cdot [1, 1, 0] = 1$
 $[0, 1, 0] \cdot [0, 0, 1] = 0; [0, 1, 0] \cdot [1, 0, 0] = 0$
 $[0, 1, 0] \cdot [0, 1, 1] = 1; [0, 1, 0] \cdot [0, 0, 0] = 0$

[0, 0, 1, 1, 0, 0]

25

25

Example: 1D Convolutional Layer (cont'd)

- Define a one-dimensional input that has eight elements all with the value of 0, with a two element bump in the middle with the values 1 below

$[0, 0, 0, 1, 1, 0, 0, 0]$

- Input to Keras must be three dimensional for a 1D convolutional layer
 - Dimension 1 refers to number of input sample: 1
 - Dimension 2 refers to the length of sample: 8
 - Dimension 3 refers to the number of channels in sample: 1

- Therefore, the shape of the input array will be $[1, 8, 1]$.

```
# define input data
data = asarray([0, 0, 0, 1, 1, 0, 0, 0])
data = data.reshape(1, 8, 1)
```

26

26

Example: 1D Convolutional Layer(cont'd)

- Define a model that expects input samples to have the shape $[8, 1]$.
- The model will have a single kernel with the shape of 3, or three elements wide.
- Keras refers to the shape of the kernel as the *kernel_size*.


```
# create model
model = Sequential()
model.add(Conv1D(1, 3, input_shape=(8, 1)))
```
- By default, a kernel in a convolutional layer is initialized with random weights. Here manually specify the weights for the single kernel that is capable of detecting bumps

27

27

Example: 1D Convolutional Layer(cont'd)

- The three element kernel are defined as follows

$[0, 1, 0]$

- The convolutional layer also has a bias input value that also requires a weight that we will set to zero.
- Therefore, we can force the weights of our one-dimensional convolutional layer to use our handcrafted filter as follows:

```
# define a vertical line detector
weights = [asarray([[[[0]], [[1]], [[0]]]), asarray([0.0])]
# store the weights in the model
model.set_weights(weights)
```

28

28

Other ConvNet Architectures

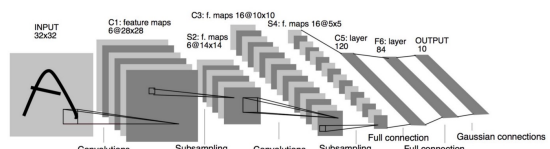
- Convolutional Neural Networks have been around since early 1990s. More other architectures can be found in
 - <https://medium.com/towards-data-science/neural-network-architectures-156e5bad51ba>
 - <http://cs231n.github.io/convolutional-networks/>
- LeNet**
 - In the years from late 1990s to early 2010s convolutional neural network were in incubation.
 - As more data and computing power became available, tasks that convolutional neural networks could tackle became more and more interesting.
- AlexNet**
 - In 2012, Alex Krizhevsky (and others) released

29

29

LeNet5

- It was published in 1994, and this is one of the very first convolutional neural networks, and what propelled the field of Deep Learning.
- This pioneering work by Yann LeCun was named LeNet5 after many previous successful iterations since the year 1988.



30

30

LeNet5 Features

- ▶ A convolutional neural network uses sequence of 3 layers: **convolution, pooling, non-linearity** → This may be the key feature of Deep Learning for images since their paper
 - ▶ use convolution to extract spatial features
 - ▶ subsample using spatial average of maps
 - ▶ non-linearity in the form of tanh or sigmoids
 - ▶ multi-layer neural network (MLP) as final classifier
 - ▶ sparse connection matrix between layers to avoid large computational cost
- ▶ In overall this network was the origin of much of the recent architectures, and a true inspiration for many people in the field.

31

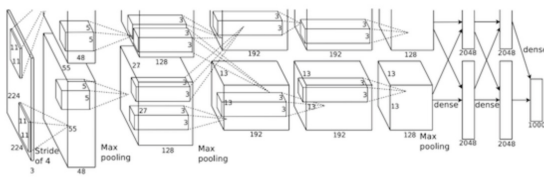
AlexNet

- ▶ In 2012, Alex Krizhevsky released AlexNet which was a deeper and much wider version of the LeNet and won by a large margin the difficult ImageNet competition.
- ▶ AlexNet scaled the insights of LeNet into a much larger neural network that could be used to learn much more complex objects and object hierarchies. The contribution of this work were:
 - ▶ use of rectified linear units (ReLU) as non-linearities
 - ▶ use of dropout technique to selectively ignore single neurons during training, a way to avoid overfitting of the model
 - ▶ overlapping max pooling, avoiding the averaging effects of average pooling
 - ▶ use of GPUs NVIDIA GTX 580 to reduce training time

32

AlexNet (cont'd)

- ▶ The success of AlexNet started a small revolution. Convolutional neural networks were now the workhorse of Deep Learning, which became the new name for "large neural networks that can now solve useful tasks".



33